

day7

Cell 1 (markdown)

Diabetes Risk Predictor: Logistic Regression vs. Decision Tree

Cell 2 (markdown)

Setup and Data Loading

To download the dataset from Kaggle, you need to set up your Kaggle API credentials. Follow these steps:

1. Go to Kaggle and log in.
2. Click on your profile picture, then 'My Account'.
3. Scroll down to the 'API' section and click 'Create New API Token'. This will download a `kaggle.json` file.
4. In Google Colab, click on the folder icon on the left sidebar to open the file browser.
5. Click on the 'Upload to session storage' icon (looks like an upward arrow) and upload the `kaggle.json` file you just downloaded.
6. Run the following cells to install the `kaggle` library, move the `kaggle.json` file to the correct directory, and download the dataset.

Cell 3 (code)

```
# Install the Kaggle library
!pip install kaggle
```

Cell 4 (code)

```
# Create .kaggle directory and move kaggle.json
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

Cell 5 (code)

```
# Download the dataset
# The dataset is 'diabetes-health-indicators-dataset' by alexteboul
!kaggle datasets download -d alexteboul/diabetes-health-indicators-dataset

# Unzip the downloaded file
import zipfile
import os

with zipfile.ZipFile('diabetes-health-indicators-dataset.zip', 'r') as zip_ref:
    zip_ref.extractall('.')

# Verify the extracted files
print(os.listdir('.'))
```

Cell 6 (code)

```
import pandas as pd

# Load the dataset into a pandas DataFrame
df = pd.read_csv('diabetes_binary_health_indicators_BRFSS2015.csv')

# Display the first 5 rows of the DataFrame
print('First 5 rows of the dataset:')
display(df.head())
```

Cell 7 (code)

```
# Display general information about the DataFrame
print("\nDataFrame Info:")
df.info()

# Display descriptive statistics of the DataFrame
print("\nDescriptive Statistics:")
display(df.describe())
```

Cell 8 (markdown)

Model Training and Evaluation

We will now train two classification models: Logistic Regression and Decision Tree. For each model, we will train it on the training data and then evaluate its performance on the testing data.

Cell 9 (markdown)

1. Logistic Regression

Cell 10 (code)

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Initialize and train the Logistic Regression model
log_reg_model = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)
log_reg_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_lr = log_reg_model.predict(X_test)
y_proba_lr = log_reg_model.predict_proba(X_test)[:, 1]

# Evaluate the model
accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)
roc_auc_lr = roc_auc_score(y_test, y_proba_lr)
```

```

conf_matrix_lr = confusion_matrix(y_test, y_pred_lr)

print("--- Logistic Regression Model Performance ---")
print(f"Accuracy: {accuracy_lr:.4f}")
print(f"Precision: {precision_lr:.4f}")
print(f"Recall: {recall_lr:.4f}")
print(f"F1-Score: {f1_lr:.4f}")
print(f"ROC AUC Score: {roc_auc_lr:.4f}")

plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix_lr, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

```

Cell 11 (markdown)

2. Decision Tree

Cell 12 (code)

```

from sklearn.tree import DecisionTreeClassifier

# Initialize and train the Decision Tree Classifier model
dec_tree_model = DecisionTreeClassifier(random_state=42)
dec_tree_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_dt = dec_tree_model.predict(X_test)
y_proba_dt = dec_tree_model.predict_proba(X_test)[:, 1]

# Evaluate the model
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)
roc_auc_dt = roc_auc_score(y_test, y_proba_dt)
conf_matrix_dt = confusion_matrix(y_test, y_pred_dt)

print("--- Decision Tree Model Performance ---")
print(f"Accuracy: {accuracy_dt:.4f}")
print(f"Precision: {precision_dt:.4f}")
print(f"Recall: {recall_dt:.4f}")
print(f"F1-Score: {f1_dt:.4f}")
print(f"ROC AUC Score: {roc_auc_dt:.4f}")

plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix_dt, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix - Decision Tree')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')

```

```
plt.show()
```

Cell 13 (markdown)

Model Comparison

Let's compare the performance metrics of both models side-by-side.

Cell 14 (code)

```
comparison_df = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC AUC Score'],
    'Logistic Regression': [accuracy_lr, precision_lr, recall_lr, f1_lr, roc_auc_lr],
    'Decision Tree': [accuracy_dt, precision_dt, recall_dt, f1_dt, roc_auc_dt]
})

display(comparison_df)
```

Cell 15 (markdown)

Data Preprocessing

We will now prepare the data for training. This involves separating features (input) from the target variable (output) and then splitting the dataset into training and testing sets to evaluate our models effectively.

Cell 16 (code)

```
from sklearn.model_selection import train_test_split

# Separate features (X) and target (y)
# The target variable is 'Diabetes_binary'
X = df.drop('Diabetes_binary', axis=1)
y = df['Diabetes_binary']

# Split the data into training and testing sets
# We'll use a 70/30 split, with a random state for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print(f"\nProportion of positive class in y_train: {y_train.mean():.2f}")
print(f"Proportion of positive class in y_test: {y_test.mean():.2f}")
```